Projeto React 02

Prof. Me. Deivison S. Takatu

deivison.takatu@edu.senai.br

SENAI

Sumário

- State e useState: estado e atualização da interface.
- Formulários: entrada e validação de dados.
- useEffect: sincronização com sistemas externos.
- Consumo de APIs: busca e exibição de dados.
- React Router: páginas e navegação por URLs.
- Atividade

Links Complementares

- Conceitos: https://github.com/deivisontakatu/aula-react
- Exemplo: https://github.com/deivisontakatu/projeto-react
- Aplicação: https://projeto-react-five-swart.vercel.app/

SENAI

STATE E USESTATE

Alternando mensagens de carregamento

- state é uma informação que o componente guarda, como um contador ou uma mensagem.
- Permite atualizar a interface conforme as ações do usuário.
- Quando o estado muda, o React renderiza novamente o componente para atualizar a tela.

Exemplo: ao clicar em um botão, um contador aumenta de valor e a tela exibe o novo número.

SENAI

Por que uma variável comum não basta?

- Variáveis comuns guardam valores, mas não atualiza a tela automaticamente.
- Alterar uma variável não faz o React renderizar o componente novamente.
- Quando o componente é executado novamente, as variáveis locais são recriadas.
- O state permite guardar valores entre renderizações e atualizar a interface quando eles mudam.

Por que uma variável comum não basta?

```
function Contador() {
  let valor = 0;
  function incrementar() {
    valor = valor + 1;
  }
  return (
    <button onClick={incrementar}>
      Valor: {valor}
    </button>
  );
}
```

Nesse exemplo, clicar no botão altera a variável, mas não solicita uma nova renderização.

SENAI

O que são Hooks no React?

- Hooks são funções especiais que permitem utilizar recursos do React em componentes funcionais.
- Permitem trabalhar com estado e outros recursos da aplicação.
- Seus nomes começam com use, como useState e useEffect.
- Devem ser utilizados no início do componente ou de outros Hooks, seguindo as regras do React.

Exemplo: o useState permite guardar o valor de um contador e atualizar a interface quando ele muda.

SENAI

Conhecendo o Hook useState

- useState permite guardar informações que podem mudar durante o uso do componente.
- Retorna dois elementos: o valor atual e a função para atualizá-lo.
- Recebe um valor inicial, como 0 para um contador.
- Deve ser importado do React.

```
import { useState } from "react";
const [contador, setContador] = useState(0);
```

SENAI

Conhecendo o Hook useState

```
import { useState } from "react";
function Contador() {
  const [contador, setContador] = useState(0);
  return (
    <section>
      <p>Valor: {contador}</p>
      <button onClick={() => setContador(contador + 1)}>
        Incrementar
      </button>
    </section>
  );
}
```

SENAI

FORMULÁRIOS

SENAI

Formulários

- Permitem que o usuário informe dados para a aplicação.
- Em React, utilizam campos HTML, eventos e estado.
- Podem ser usados para cadastros, buscas, login e preferências.
- O envio pode validar os dados e exibir uma mensagem ao usuário.

Exemplo: um cadastro que recebe nome e e-mail e exibe uma confirmação após o envio.

SENAI

O que são campos controlados?

- São campos cujo valor é controlado pelo estado do React.
- A propriedade value define o valor exibido no campo.
- O evento onChange atualiza o estado conforme o usuário digita.

```
const [nome, setNome] = useState("");
<input value={nome} onChange={e => setNome(e.target.value)} />
```

SENAI

O que são campos controlados?

```
import { useState } from "react";
function CampoNome() {
  const [nome, setNome] = useState("");
  return (
    <label>
      Nome
      <input
        value={nome}
        onChange={(evento) => setNome(evento.target.value)}
      />
    </label>
  );
}
```

SENAI

Como atualizar um campo?

- onChange detecta quando o usuário digita.
- e.target.value obtém o texto digitado.
- setNome() atualiza o estado com o novo valor.
- Fluxo: digitação → evento → atualização do estado.

```
const [nome, setNome] = useState("");
<input
  value={nome}
  onChange={(e) => setNome(e.target.value)}
/>
```

Como funciona o envio?

- onSubmit executa uma função quando o formulário é enviado.
- A função pode validar os dados ou exibir uma mensagem.
- preventDefault() evita o recarregamento da página.
- O envio pode ser tratado diretamente pelo React.

SENAI

O que são campos controlados?

```
function enviar(e) {
  e.preventDefault();
  alert("Formulário enviado!");
}

<form onSubmit={enviar}>
  <button type="submit">Enviar</button>
</form>
```

SENAI

Para que serve preventDefault()?

- Impede que o navegador execute uma ação padrão.
- Em formulários, evita o recarregamento automático da página.
- Permite que o React controle o que acontece após o envio.

```
function enviar(e) {
  e.preventDefault(); // Evita recarregar a página
  alert("Enviado com sucesso!");
}
```

SENAI

USEEFFECT

SENAI

O que são efeitos no React?

- Efeitos permitem executar ações fora da renderização da interface.
- São usados quando o componente precisa interagir com recursos externos.
- Exemplos: alterar o título da aba, usar temporizadores e buscar dados de APIs.

Exemplo: atualizar o título da aba do navegador conforme a página exibida.

SENAI

O que é useEffect?

- useEffect é um Hook utilizado para sincronizar um componente com sistemas externos.
- Recebe uma função que contém o efeito.
- Pode receber uma lista de dependências para controlar quando o efeito precisa ser executado novamente.

```
useEffect(() => {
  document.title = "Aula React";
}, []);
```

SENAI

Exemplo: atualizar o título da página

- O efeito sincroniza o título da aba com o valor recebido por props.
- Quando titulo muda, o efeito é executado novamente.
- A dependência precisa refletir os valores utilizados pelo efeito.

```
useEffect(() => {
  document.title = titulo;
}, [titulo]);
```

Aplicação: atualizar o título da aba conforme a página ou seção acessada.

SENAI

O que é a limpeza de um efeito?

- Alguns efeitos criam recursos que precisam ser liberados.
- A função de limpeza é retornada pelo próprio useEffect.
- Ela evita que eventos ou temporizadores continuem ativos indevidamente.
- A limpeza ocorre antes de uma nova configuração do efeito quando suas dependências mudam e quando o componente deixa de ser utilizado.

SENAI

O que é a limpeza de um efeito?

```
useEffect(() => {
  window.addEventListener("resize", atualizar);
  return () => {
    window.removeEventListener("resize", atualizar);
  };
}, []);
```

Conceito-chave: tudo o que for configurado e precisar ser encerrado deve ter uma limpeza adequada.

SENAI

CONSUMO DE API

SENAI

O que é uma API?

- API (Application Programming Interface) permite a comunicação entre sistemas.
- Em aplicações React, possibilita buscar e enviar dados para serviços externos.
- Exemplos: produtos, usuários, publicações e indicadores.

```
const resposta = await fetch(
  "https://jsonplaceholder.typicode.com/posts"
);
```

SENAI

Buscando dados com fetch

- fetch() realiza requisições HTTP.
- A resposta é assíncrona: precisamos aguardar seu retorno.
- response.json() interpreta os dados JSON.
- Verifique response.ok para identificar erros HTTP.

```
const response = await fetch(url);
if (!response.ok) throw new Error("Erro");
const dados = await response.json();
```

SENAI

O papel do useEffect

- useEffect pode iniciar uma busca de dados quando o componente é exibido.
- O array vazio [] indica que o efeito será executado na montagem do componente.
- useState armazena os dados recebidos e atualiza a interface quando eles mudam.

SENAI

O que é a limpeza de um efeito?

- A limpeza permite desfazer ações iniciadas por um efeito.
- É útil para cancelar temporizadores, remover eventos e interromper conexões.
- Em buscas de dados, ajuda a evitar atualizações desnecessárias quando o componente deixa de ser exibido.

Exemplo: retornar uma função dentro do useEffect para cancelar uma ação ou liberar um recurso quando necessário.

SENAI

O que é a limpeza de um efeito?

```
useEffect(() => {
  async function carregar() {
    const resposta = await fetch(url);
    const dados = await resposta.json();
    setPublicacoes(dados);
  }
  carregar();
}, []);
```

SENAI

Enviando dados com POST

- O método POST permite enviar informações para uma API.
- headers informa o formato do conteúdo.
- JSON.stringify() transforma o objeto JavaScript em JSON.
- Sempre verifique se a resposta foi bem-sucedida.

```
await fetch("/api/publicacoes", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Aprendendo React" })
});
```

SENAI

Erros e cuidados importantes

- Trate erros de rede e verifique o status HTTP.
- Considere carregamento, estado vazio e indisponibilidade do serviço.
- Evite requisições repetidas por dependências incorretas no useEffect.
- Não exponha chaves secretas no código do navegador.
- Respeite autenticação, limites de uso e políticas de CORS.

SENAI

Erros e cuidados importantes

```
try {
  const resposta = await fetch(url);

  if (!resposta.ok) throw new Error("Erro na consulta");

  const dados = await resposta.json();
} catch (erro) {
  console.error(erro);
}
```

SENAI

REACT ROUTER

SENAI

Por que precisamos de roteamento?

- Uma aplicação React pode ter várias telas, como início, catálogo, detalhes e sobre.
- O roteamento associa caminhos da URL aos componentes exibidos.
- Organiza a navegação e evita concentrar toda a lógica em um único componente.

```
// Cada caminho pode exibir uma página diferente
<Route path="/" element={<Inicio />} />
<Route path="/sobre" element={<Sobre />} />
```

SENAI

O que é React Router?

- É uma biblioteca utilizada para organizar a navegação em React.
- Associa URLs a componentes.
- Permite navegar entre páginas sem recarregar todo o documento a cada clique.

```
import {
  BrowserRouter,
  Routes,
  Route
} from "react-router-dom";
```

SENAI

Conceitos fundamentais

- Rota: regra que associa um caminho a uma interface.
- URL: endereço da página atual.
- Link: elemento para navegar entre rotas.
- Parâmetro: parte variável do caminho.
- Rota aninhada: rota organizada dentro de uma estrutura pai.

```
<Route path="/conceitos" element={<Conceitos />} />
<Link to="/conceitos">Ver conceitos</Link>
```

Configurando as rotas

- BrowserRouter envolve a aplicação.
- Routes agrupa as rotas disponíveis.
- Route define o caminho e o componente correspondente.

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Inicio />} />
    <Route path="/sobre" element={<Sobre />} />
  </Routes>
</BrowserRouter>
```

Navegação com Link

- O componente Link permite navegar entre rotas.
- A propriedade to define o destino.
- É preferível a um link tradicional quando a navegação ocorre dentro da aplicação React.

```
import { Link } from "react-router-dom";
<nav>
  <Link to="/">Início</Link>
  <Link to="/sobre">Sobre</Link>
</nav>
```

SENAI

Parâmetros de rota

- Parâmetros representam partes variáveis de uma URL.
- São úteis para páginas de detalhes.
- useParams() permite acessar esses valores.

```
// Rota: /conceitos/:id
const { id } = useParams();
return <h1>Conceito: {id}</h1>;
```

SENAI

Atividade

Desenvolvam uma aplicação React que utilize todos os conceitos estudados nesta aula, incluindo State e useState, Formulários, useEffect, Consumo de APIs, React Router

Criem um repositório público no GitHub e um README contendo a descrição, as funcionalidades, e os conceitos utilizados.

Referências

1. SOUZA, Natan. **Bootstrap 4: conheça a biblioteca front-end mais utilizada no mundo.** São Paulo: Casa do Código, 2018. E-book. Disponível em: https://plataforma.bvirtual.com.br.

2. MACHADO, Kheronn Khennedy. **Angular 11 e Firebase: construindo uma aplicação integrada com a plataforma do Google.** São Paulo: Casa do Código, 2021. E-book. Disponível em: https://plataforma.bvirtual.com.br. Acesso em: 13 maio 2025.

3. EIS, Diego. **Guia Front-end: o caminho das pedras para ser um dev front-end.** São Paulo: Casa do Código, 2015. E-book. Disponível em: https://plataforma.bvirtual.com.br.

4. GONÇALVES, Edson. **Desenvolvendo aplicações Web com JSP, Servlets, JavaServer Faces, Hibernate, EJB 3 Persistence e Ajax.** Rio de Janeiro: Ciência Moderna, c2007.

Referências

5. HARTCOPP, Patrícia Ferreira. **Métrica Web.** São Paulo: Contentus, 2020. E-book (94 p.). Disponível em: https://plataforma.bvirtual.com.br/Acervo/Publicacao/185191. Acesso em: 30 abr. 2024.

6. NIEDERAUER, Juliano. **Desenvolvendo Websites com PHP: aprenda a criar Websites dinâmicos e interativos com PHP e banco de dados.** 3. ed. São Paulo: Novatec, 2017.

7. PREECE, J.; ROGERS, Y.; SHARP, H. **Design de Interação: além da interação Homem-Computador.** 3. ed. Porto Alegre: Bookman, 2013.

8. SOUSA, Roque Fernando Marcos. **Canvas HTML 5: composição gráfica e interatividade na Web.** Rio de Janeiro: Brasport, 2014. E-book (194 p.). Disponível em: https://plataforma.bvirtual.com.br/Acervo/Publicacao/160686. Acesso em: 22 jun. 2024.

Projeto React 02

Prof. Me. Deivison S. Takatu

deivison.takatu@edu.senai.br